Lab setup instructions and requirements

End Lab 00:56:53

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. Learn more.

Open Google Cloud Console

Username

student-02-9144edc4d692@

Password

1doxr57QnfZ@

GCP Project ID

qwiklabs-gcp-03-115b8cc

Use OpenAPI Tools with Conversational Agents

Lab · 1 hour · No cost · Intermediate

★★★★½ Rate Lab

This lab may incorporate AI tools to support your learning.

Overview

This lab guides you through creating an OpenAPI Tool for use by Conversational Agents.

In this lab, you will imagine working for an ice cream company, creating a chatbot that records users' email addresses and favorite flavors of ice cream for use in future promotional communications.

Note: Using an Incognito browser window is recommended for most Qwiklabs to avoid confusion between multiple Qwiklabs student and other accounts. It is particularly helpful for Qwiklabs like this one that use the Conversational agents console. If you are using Chrome, the easiest way to accomplish this is to close any Incognito windows, then right click on the **Open Google Cloud console** button at the top of this lab and select **Open link in Incognito window**.

Objectives

In this lab, you learn how to deploy:

- a Cloud Run function
- a Conversational Agent Tool with an OpenAPI spec to call the function
- a Playbook that uses the Tool

Setup and requirements

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources will be made available to you.

This Qwiklabs hands-on lab lets you do the lab activities yourself in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials that you use to sign in and access Google Cloud for the duration of the lab.

What you need

To complete this lab, you need:

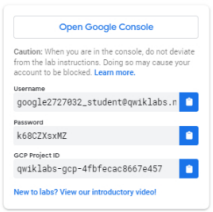
- Access to a standard internet browser (Chrome browser recommended).
- Time to complete the lab.

Note: If you already have your own personal Google Cloud account or project, do not use it for this lab.

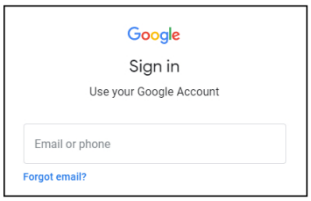
Note: If you are using a Pixelbook, open an Incognito window to run this lab.

How to start your lab and sign in to the Google Cloud Console

1. Click the **Start Lab** button. If you need to pay for the lab, a pop-up opens for you to select your payment method. On the left is a panel populated with the temporary credentials that you must use for this lab.



2. Copy the username, and then click **Open Google Console**. The lab spins up resources, and then opens another tab that shows the **Sign in** page.

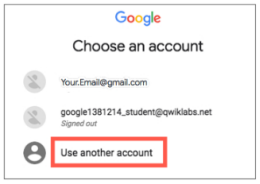


Tip: Open the tabs in separate windows, side-by-side.

Lab instructions and tasks

Overview
Objectives
Setup and requirements
Task 1. Create a BigQuery table where user data will be stored
Task 2. Create a Cloud Run Function to record the requests
Task 3. Create a Conversational Agent Tool to call the Cloud Run Function
Task 4. Create a Conversational Agent Playbook
Congratulations!

If you see the **Choose an account** page, click **Use Another Account**.



3. In the **Sign in** page, paste the username that you copied from the Connection Details panel. Then copy and paste the password.

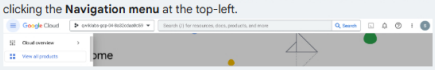
Important: You must use the credentials from the Connection Details panel. Do not use your Qwiklabs credentials. If you have your own Google Cloud account, do not use it for this lab (avoids incurring charges).

4. Click through the subsequent pages:

- Accept the terms and conditions.
- Do not add recovery options or two-factor authentication (because this is a temporary account).
- Do not sign up for free trials.

After a few moments, the Cloud Console opens in this tab.

Note: You can view the menu with a list of Google Cloud Products and Services by clicking the **Navigation menu** at the top-left.



Task 1. Create a BigQuery table where user data will be stored

In this task, you'll create a table to serve as the destination to store users' emails and favorite ice cream flavors.

1. Open a **Cloud Shell** terminal by selecting your Cloud Console window, then typing on your keyboard the **G** key and then the **S** key.

2. In your Cloud Shell terminal, paste the following to create a `favorite_flavors_schema.json` file to define the schema for a BigQuery table which will be used to record users' favorite flavorss:

```
cat > favorite_flavors_schema.json << EOF
[
  {
    "name": "email",
    "type": "STRING",
    "mode": "NULLABLE"
  },
  {
    "name": "favorite_flavor",
    "type": "STRING",
    "mode": "NULLABLE"
  }
]
EOF
```

2. Create the BigQuery dataset and table by running the following in Cloud Shell:

```
bq --location=US mk -d users
bq mk -t users.favorite_flavors favorite_flavors_schema.json
```

3. Click the **Authorize** button when prompted to authorize Cloud Shell.

4. Once the dataset and table have been successfully created, you can close the Cloud Shell panel by clicking the **X** in the upper right of the terminal panel.

Task 2. Create a Cloud Run Function to record the requests

In this task, you'll create the Cloud Run function that will take requests sent as JSON and write them to the table you created above.

1. Using the search bar at the top of the Cloud Console, navigate to **Cloud Run**.

2. From the options at the top of the Cloud Run console, select **Write a Function**.

3. Under the **Configure** header, enter a **Service name** of `record-favorite-flavor`.

4. Set the **Region** to **us-central1 (Iowa)**.

5. Under the **Endpoint URL** header, copy the provided URL and paste it in a text document. You will need to access it later.

6. Set the **Runtime** to **Python 3.12**.

7. Under the **Authentication** header, selet **Require authentication**.

8. Keep the other settings as they are and select **CREATE**. The **Source** tab for the function will be loaded.

9. Rename the **Function entry point** to `record_favorite_flavor`.

10. Click the **requirements.txt** file on the left, delete its contents, and paste in the following:

```
functions-framework==3.*
google-cloud-bigquery
```

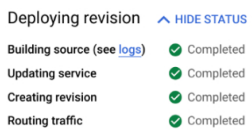
12. Click the **main.py** file on the left, delete its contents, and paste in the following code. This function will take the travel request details provided to a POST request as JSON and write those values to a new row in the BigQuery table you created earlier:

```
import functions_framework
from google.cloud import bigquery

@functions_framework.http
def record_favorite_flavor(request):
    """Writes user emails and favorite flavors to BigQuery."""

    request_json = request.get_json(silent=True)
    request_args = request.args
    print("JSON:" + str(request_json))
    print("args:" + str(request_args))

    bq_client = bigquery.Client()
    table_id = "YOUR_PROJECT_ID.users.favorite_flavors"

    row_to_insert = [
        {"email": request_json["email"],
        "favorite_flavor": request_json["favorite_flavor"]
        },
    ]

    errors = bq_client.insert_rows_json(table_id, row_to_insert)
# Make an API request.
    if errors == []:
        return {"message": "New row has been added."}
    else:
        return {"message": "Encountered errors while inserting
rows: {}".format(errors)}
```

13. Update `YOUR_PROJECT_ID` in the pasted code to your Qwiklabs project ID.

14. Click **SAVE AND REDEPLOY**.

15. Wait until the the **Deploying revision** activities all show a **Completed** status message.



15. Click **TEST** at the top of the Cloud Run console.

16. Paste these values into the **Configure triggering event** test input field:

```
{
  "email": "ice_cream_fan@fans.com",
  "favorite_flavor": "vanilla"
}
```

14. At the bottom of the testing pane, click **TEST IN CLOUD SHELL**.

15. The Cloud Shell terminal window will open, and a **curl** command will be prepared for you to call your function. Select the Terminal window and press **enter** or **return** on your keyboard to send the command.

16. You should see a successful execution response:

```
{"message":"New row has been added."}
```

17. Close the Cloud Shell Terminal by pressing the **X** in the upper right of the Cloud Shell panel.

18. Open a new browser tab (or if you are using Chrome, duplicate your current browser tab by right-clicking on it and selecting *Duplicate*). In the new tab, navigate to **BigQuery**.

19. In the BigQuery Explorer pane, select your project ID.

20. Select the **users** dataset.

21. Select the **favorite_flavors** table.

22. Select the **PREVIEW** tab to see travel requests that have been recorded. Keep this tab open to return and check for new rows as you work through the other components in this lab.

Task 3. Create a Conversational Agent Tool to call the Cloud Run Function

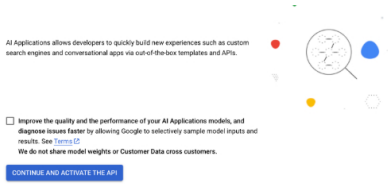
Now you can create a Tool in the Conversational Agent console that can call the Cloud Run function. Your conversational agent will be able to use this Tool to record users' inputs.

1. At the top of the Google Cloud Console, search for **Dialogflow API** and select it.

2. Click **Enable**.

3. Navigate to **AI Applications** by searching for it at the top of the console.

4. Click on **CONTINUE AND ACTIVATE THE API**.



5. Click **+ Create App**.
6. For an app type, find the **Conversational agent** card and select **Create**. This will open the **Conversational Agents** console in a new tab.
7. In the **Get started with Conversational Agents** pane, select **Build your own**.
8. For your agent's **Display name**, use `Favorite Flavor Agent`.
9. Set the **location** to **global**.
10. Keep the **Conversation start** option **Playbook** selected.
11. Click **Create**.
12. You will be taken to your starting Playbook. Dismiss any instructional pop-ups. Before you create the Playbook, you will create a Tool that the playbook can use to call the Cloud Run function you created. From the left-hand navigation menu, select **Tools**.
13. Click **+ Create**.
14. For **Tool name** enter `Record Favorite Flavor`.
15. Keep the **Type** set to **OpenAPI**.
16. For a **Description**, enter `Used to record emails and favorite flavors.`
17. For **Schema**, keep the type **YAML** selected, and paste the following into the text box. This OpenAPI spec describes an API with:

- a server url set to call your Cloud Run function
- a default path for your Cloud Run function (`/`) that accepts POST requests that include JSON using a schema called FavoriteFlavor
- a definition of that FavoriteFlavor schema to include the fields you defined in your BigQuery schema at the start of this lab:

```
openapi: 3.0.0
info:
  title: Favorite Flavors API
  version: 1.0.0
servers:
  - url: 'YOUR_CLOUD_RUN_FUNCTION_URL'
paths:
  /:
    post:
      summary: Record a user's favorite flavor
      operationId: recordFavoriteFlavor
      requestBody:
        description: Favorite flavor to add
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/FavoriteFlavor'
      responses:
        '200':
          description: Success
components:
  schemas:
    FavoriteFlavor:
      type: object
      properties:
        email:
          type: string
        favorite_flavor:
          type: string
```

18. Replace YOUR_CLOUD_RUN_FUNCTION_URL in the spec above with your Cloud Run function's URL that you copied earlier.
19. Click **Save** at the top of the Tools pane.
20. To allow the Tool to invoke the Cloud Run function, switch to your browser tab displaying the Cloud Console (not the Conversational Agents console) and use the search bar at the top of the console to navigate to **IAM**.
21. Check the checkbox to **Include Google-provided role grants**.
22. Find the row for the **Dialogflow Service Agent** and click the pencil edit icon on its row.
23. Click **+ Add Another Role**.
24. In the **Select a role** field, enter `Cloud Run Invoker`.
25. Click **SAVE**.

Task 4. Create a Conversational Agent Playbook

Now that the Tool is ready, you can create the conversational agent that can receive user messages in natural language and use the Tool to write them to the BigQuery table via the Cloud Run function.

1. In the browser tab with the Conversational Agents console, use the left-hand navigation menu to select **Playbooks**.

2. Select the **Default Generative Playbook** that has already been created.
3. Change the **Playbook name** to `Ask for Favorite Flavor`.
4. For a **Goal**, enter: `Get users' emails and favorite flavor.`
5. For **Instructions**, paste the following into the text field:

```
- Greet a user and tell them a fact about ice cream.
- Ask them to share their favorite flavor of ice cream and their
email address to receive special promotions they might enjoy.
- When they have provided both, record them with ${TOOL:Record
Favorite Flavor}
- Thank the user and suggest that they try our new Chocolate
Strawberry Swirl flavor.
```

7. Click **Save** at the top of the Playbook pane.
8. In the upper right of the Conversational Agents console, click the **Toggle simulator** chat button ( ) to preview the conversational agent.
9. In the **Enter text** input box at the bottom of this panel, start the chat with a simple `hi`.
10. Within 1 or 2 messages, the bot should ask you for an email address and/or favorite flavor. Reply to its questions. For an email address, you can use `ice@example.com` and for a favorite flavor, you can use your real favorite, or say `mint chocolate chip`.
11. In the simulator, you should see a card indicating that the bot used the Record Favorite Flavor Tool, and you can click on it to see the Tool input and response:

12. Your chat should end with a statement similar to: `You should definitely try our new Chocolate Strawberry Swirl flavor! It's been a huge hit.
13. To confirm that user inputs are being recorded, return to the browser tab displaying your BigQuery **favorite_flavors** table. On the **PREVIEW** tab, click the **Refresh icon** to view the latest data.

Congratulations!

In this lab, you've learned how to configure an OpenAPI Tool to allow a Conversational Agent Playbook to call a Cloud Run function. More specifically, you've learned to deploy:

- a Cloud Run function
- a Conversational Agent Tool with an OpenAPI spec to call the function
- a Playbook that uses the Tool

Manual Last Updated April 03, 2025

Lab Last Tested April 03, 2025